



Computer Vision

Image classification using neural network and convolutional neural network approaches

by

Alpaycan CABBAR

School of Electronics,
Electrical Engineering and Computer Science
Queen's University Belfast

April 2026

Table of Contents

1. Introduction	3
2. Methodology.....	4
2.1 Dataset Preparation	4
2.2 Project 1 Hand-Crafted Feature Baseline	4
2.3 Plain Neural Network.....	4
2.4 Simple CNN	5
2.5 Resnet18 with Transfer Learning.....	6
2.6 Hyperparameter Selection and Training Settings.....	6
2.7 Evaluation	8
3. Result	8
4. Discussion	12
5. Conclusion.....	14
References	15

List of Figures

Figure 1. Accuracy improvement between the main classification stages	9
Figure 2. Accuracy comparison bar chart of all methods	9
Figure 3. Confusion matrix of the plain Neural Network	10
Figure 4. Confusion matrix of the simple CNN	11
Figure 5. Confusion matrix of ResNet18 with transfer learning	11

List of Tables

Table 1. Test accuracy of Project 1 hand-crafted feature methods	4
Table 2. Selected epoch results of the plain neural network	5
Table 3. Selected epoch results of the simple CNN	6
Table 4. Selected epoch results of the ResNet18 with transfer learning.....	6
Table 5. Model Comparison	7
Table 6. Final test accuracy of all methods	8
Table 7. Advantages and disadvantages of the main methods.....	14

1. Introduction

Image classification is an important task in computer vision. The goal is to put an input image in the right group. Deep learning techniques have become very good at classifying images recently because they can learn useful visual patterns directly from data. Convolutional neural networks work well because they can keep the spatial structure of the image while learning. Deeper models like ResNet have made this even better.

The main goal of this project is to use the TinyImageNet100 dataset to classify images. Project 2 used the same dataset as Project 1 so that the comparison would be fair, as required by the coursework. For this project, 15 classes were chosen, and the performance of different methods was compared on the same train and test split. The report includes a hand-crafted feature baseline derived from Project 1, a basic neural network, a straightforward convolutional neural network, and an enhanced ResNet18 model leveraging transfer learning.

The primary objective of this project is to investigate the impact of methodological choices on classification performance. Project 1 employed conventional hand-crafted feature extraction techniques, including SIFT, ORB, Bag of Words, and Fisher Vector. Conversely, Project 2 concentrated on neural network approaches that directly learn features from image data. The first step was to train a plain neural network. Then, a CNN was built from the ground up and tested. Finally, ResNet18 with transfer learning was used to make the CNN work better. SIFT is a local descriptor that is used a lot, and ResNet is a residual deep network that makes it easier to train deeper models.

Overall, this project makes it easy to see the differences between old feature-based methods and new deep learning methods for classifying images. Using the same chosen dataset at every stage makes it easier to see how each method got better and why the CNN-based methods worked better.

2. Methodology

2.1 Dataset Preparation

The TinyImageNet100 dataset was used to sort images in this project. To make the comparison between Project 1 and Project 2 fair, the same 15 classes were used in both projects. After loading all the class folders from the dataset, only these classes were kept for the tests. The first 400 images were used to train each class, and the next 100 images were used to test each class. This made sure that all the methods used in the project were used equally.

After making the lists of train and test images, the names of the classes were changed into numbers. The models couldn't work with text labels directly. A dictionary was made to connect each class name to an integer value. This label mapping was used in all the training and testing that came after.

2.2 Project 1 Hand-Crafted Feature Baseline

I used the hand-made feature methods from Project 1 as a starting point for my comparisons. I tried three feature pipelines: ORB + Bag of Words + LinearSVC, SIFT + Bag of Words + LinearSVC, and Fisher Vector (SIFT + GMM) + LinearSVC. SIFT is a popular way to describe local features, and Bag of Words and Fisher Vector are two ways to encode features that can be used to represent local image descriptors for classification.

Fisher Vector (SIFT + GMM) + LinearSVC was the best of the hand-crafted methods, so it was used as the main result for Project 1 in the final comparison. This method worked better than ORB + BoW and SIFT + BoW, but it was still not as accurate as the deep learning methods used in Project 2.

Table 1. Test accuracy of Project 1 hand-crafted feature methods

Method	Test Accuracy (%)
<i>A1: ORB + BoW + LinearSVC</i>	7.26
<i>A1: SIFT + BoW + LinearSVC</i>	22.00
<i>A1: Fisher Vector + LinearSVC</i>	22.93

2.3 Plain Neural Network

The images were first changed using PyTorch transforms for the neural network method. The training images were resized to 32×32 pixels. Then, random horizontal flipping and slight random rotations were applied. After that, the images were converted into tensors, normalized, and finally flattened into a single vector. The test images underwent the same resizing, tensor conversion, normalization, and flattening process, remaining unaltered otherwise. Compose provides a means to chain together transformations within PyTorch. To prepare data for training and testing, Dataset and DataLoader are commonly employed.

A custom dataset class was then created to manage image loading, apply the necessary changes, and return the processed images with their corresponding labels. Subsequently,

DataLoader objects were created for both the training and test datasets, each using a batch size of 32. The test data was kept in a fixed order to allow for evaluation, while the training data was shuffled to help with the learning process.

The plain neural network had fully connected layers because the input image was flattened before it went into the model. The final version of the network had three hidden layers that were 512, 256, and 128 units wide, and then an output layer for the 15 classes. To improve training stability and reduce overfitting, batch normalization, ReLU activation functions, and dropout were integrated between the model's layers. The model's training utilized CrossEntropyLoss, coupled with the Adam optimizer, a learning rate of 0.0005, and a weight decay of 1e-4. Several configurations were investigated, encompassing modifications to image size, epoch count, and regularization methods; the final configuration was chosen based on its enhanced performance compared to earlier iterations.

Table 2. Selected epoch results of the plain neural network

Epoch	Loss	Training Accuracy (%)
1	2.5156	18.08
5	2.1497	29.88
10	2.0355	34.30
15	1.9699	36.28
20	1.9039	38.02
25	1.8750	38.75

Test Accuracy of the plain neural network is 40.80%

2.4 Simple CNN

The preprocessing pipeline for the simple CNN differed from the previous one. Images were resized to 64×64 pixels, converted into tensors, and standardized. To improve the model's ability to generalize and to increase the variety in the training data, I also included a random horizontal flip and a small random rotation. The CNN did not flatten the image at the start like the plain neural network did. The architecture of Convolutional Neural Networks (CNNs) is inherently suited to image processing due to their direct manipulation of image structure.

The basic CNN was constructed using three convolutional blocks. Each of these blocks incorporated a convolution layer, a ReLU activation function, and a max-pooling layer. These layers enabled the network to learn visual patterns while also reducing the image's spatial dimensions. After feature extraction, the output was flattened and then processed through a fully connected layer, a dropout layer, and the final output layer, which was designed to classify the input into one of fifteen distinct classes. The model's training incorporated CrossEntropyLoss and the Adam optimizer, employing a learning rate of 0.001 and a weight decay of 1e-4. CNNs demonstrate superior performance in image classification tasks compared to conventional fully connected networks, primarily due to their capacity to discern spatial patterns, including edges, textures, and shapes—while simultaneously preserving the inherent structure of the image.

Table 3. Selected epoch results of the simple CNN

Epoch	Loss	Training Accuracy (%)
1	2.4188	18.78
5	1.6940	44.78
10	1.3940	55.17
15	1.1934	60.93
20	1.0498	65.63
25	0.9070	69.58

Test Accuracy of the simple CNN: 58.40%

2.5 Resnet18 with Transfer Learning

To improve performance, a more complex CNN model, based on ResNet18, was used after the simpler CNN was trained. The images were resized to 224×224 pixels and normalized using the mean and standard deviation values from the pretrained ResNet models. The same custom dataset class and data loader structure were used, but with the new ResNet transformations.

ResNet18's pre-trained weights were employed, with its final fully connected layer substituted by a novel output layer designed for the 15 selected classes. This method allowed the model to keep the important image features it had previously learned and then adjust them to fit the specific needs of the current classification task. This, known as transfer learning, involves applying knowledge gained from a pre-trained model to a different task. Residual learning is the basis for ResNet models making it easier for deeper neural networks.

Table 4. Selected epoch results of the ResNet18 with transfer learning

Epoch	Loss	Training Accuracy (%)
1	0.9535	72.32
3	0.2684	92.50
6	0.0887	97.70
9	0.0425	99.10
12	0.0443	98.80
15	0.0304	99.25

Test Accuracy of ResNet18 with transfer learning: 83.73%

2.6 Hyperparameter Selection and Training Settings

To facilitate training, the input images were resized to 32×32 pixels, thereby diminishing the dimensionality of the input features. A batch size of 32 was employed to achieve a compromise between the stability of the learning process and the efficiency of training. The model architecture incorporated three hidden layers, comprising 512, 256, and 128 neurons, respectively; this configuration was selected because a more complex structure demonstrated superior performance compared to a less intricate network. Batch normalization was used after each hidden layer to make training more stable. Dropout rates of 0.5, 0.4, and 0.3 were added to reduce overfitting. The model was trained using the Adam optimizer with a learning rate of

0.0005 and weight decay of $1e-4$, because these settings gave better generalization than earlier trials. The network was trained for 25 epochs, since this provided a better balance between learning and overfitting than shorter or much longer training.

For the scratch CNN, the input images were resized to 64×64 pixels. The selected architecture aimed to retain a greater amount of visual data compared to the MLP, while remaining feasible for training purposes. A batch size of 32 was employed once more. The CNN architecture incorporated three convolutional blocks, each utilizing 32, 64, and 128 filters, succeeded by ReLU activation functions and max-pooling layers. This configuration facilitated the model's capacity to learn local image characteristics, concurrently diminishing the image dimensions. Preceding the ultimate output layer, a fully connected layer comprising 256 neurons was integrated. To reduce overfitting, a dropout rate of 0.5 was used. The model was trained using CrossEntropyLoss and the Adam optimizer, with a learning rate of 0.001 and a weight decay of $1e-4$. The training process lasted for 25 epochs. To improve generalization, data augmentation was applied during training using random horizontal flip and random rotation of 10 degrees.

For ResNet18, the input images were resized to 224×224 , which matches the expected input size of the pretrained model. A batch size of 32 was used. Data augmentation was applied to the training set using random horizontal flip and random rotation of 10 degrees, while the images were also normalized using the standard ImageNet mean and standard deviation values. ResNet18 was loaded with pretrained weights, and the final fully connected layer was replaced with a new layer containing 15 output units for the selected classes. This transfer learning approach was used because it allowed the model to start from previously learned visual features instead of learning everything from scratch. The model was trained using the Adam optimizer with a smaller learning rate of 0.0001, since pretrained networks usually need gentler updates. The model was trained for 15 epochs was enough to achieve strong performance without making training unnecessarily long.

Table 5. Model Comparison

<i>Model</i>	Input size	Batch size	Optimizer	Learning rate	Weight decay	Epochs	Regularization / extra settings
<i>MLP</i>	32×32	32	Adam	0.0005	$1e-4$	25	BatchNorm, Dropout (0.5, 0.4, 0.3), flip, rotation
<i>Scratch CNN</i>	64×64	32	Adam	0.001	$1e-4$	25	Dropout (0.5), flip, rotation
<i>ResNet18 + Transfer Learning</i>	224×224	32	Adam	0.001	—	15	Pretrained weights, ImageNet normalization, flip, rotation

2.7 Evaluation

The same fixed test set was used to test all the models so that the comparison would be fair. The primary metric for evaluation in this project was test accuracy. Along with the final accuracy values, confusion matrices were made for the plain neural network, the simple CNN, and ResNet18. I used these matrices to see how well each model predicted individual classes and to find out which classes were more likely to be mixed up with each other.

3. Result

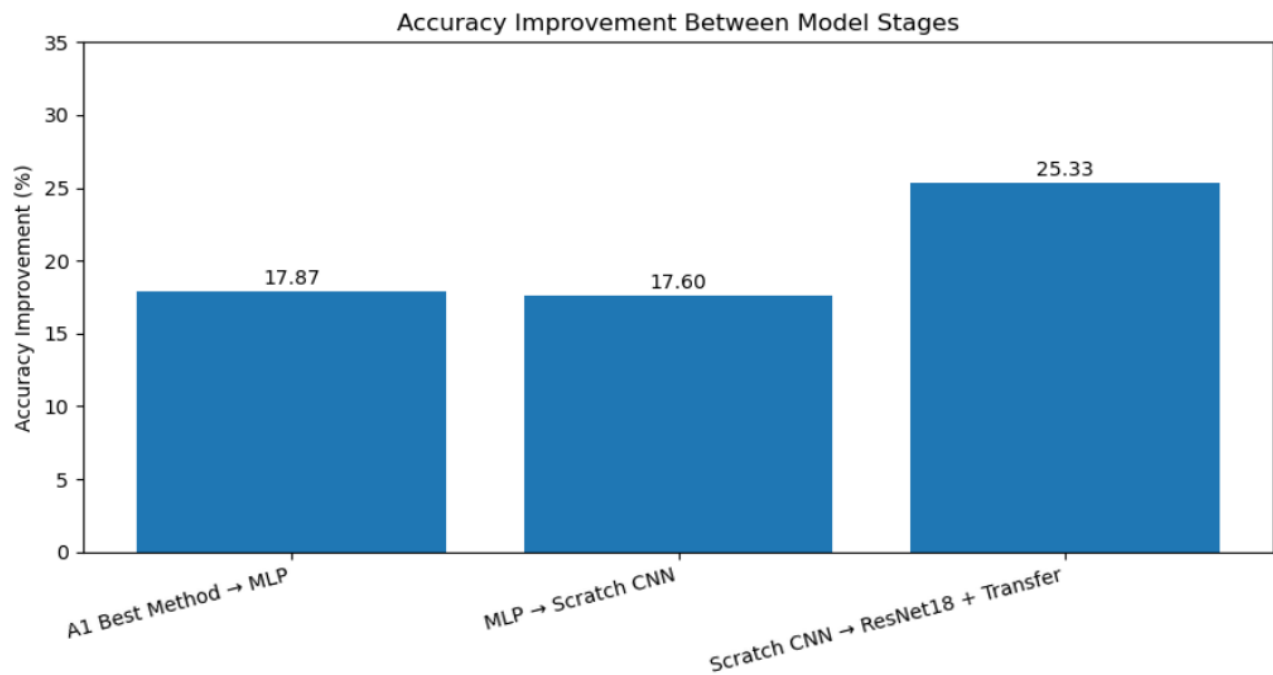
The performance of all methods was compared by using test accuracy. The Fisher Vector (SIFT + GMM) + LinearSVC method from Project 1 was the best hand-crafted feature method, with an accuracy of 22.93%. The plain neural network then made the result better, bringing the accuracy around 40.80%. Simple CNN did better than the plain neural network, getting around 58.40% of the answers right. ResNet18 with transfer learning got the best result of the whole project, with an accuracy of 83.73%.

Table 6. Final test accuracy of all methods

Method	Test Accuracy (%)
<i>A1: ORB + BoW + LinearSVC</i>	7.26
<i>A1: SIFT + BoW + LinearSVC</i>	22.00
<i>A1: Fisher Vector + LinearSVC</i>	22.93
<i>Plain Neural Network</i>	40.80
<i>Simple CNN</i>	58.40
<i>ResNet18 + Transfer Learning</i>	83.73

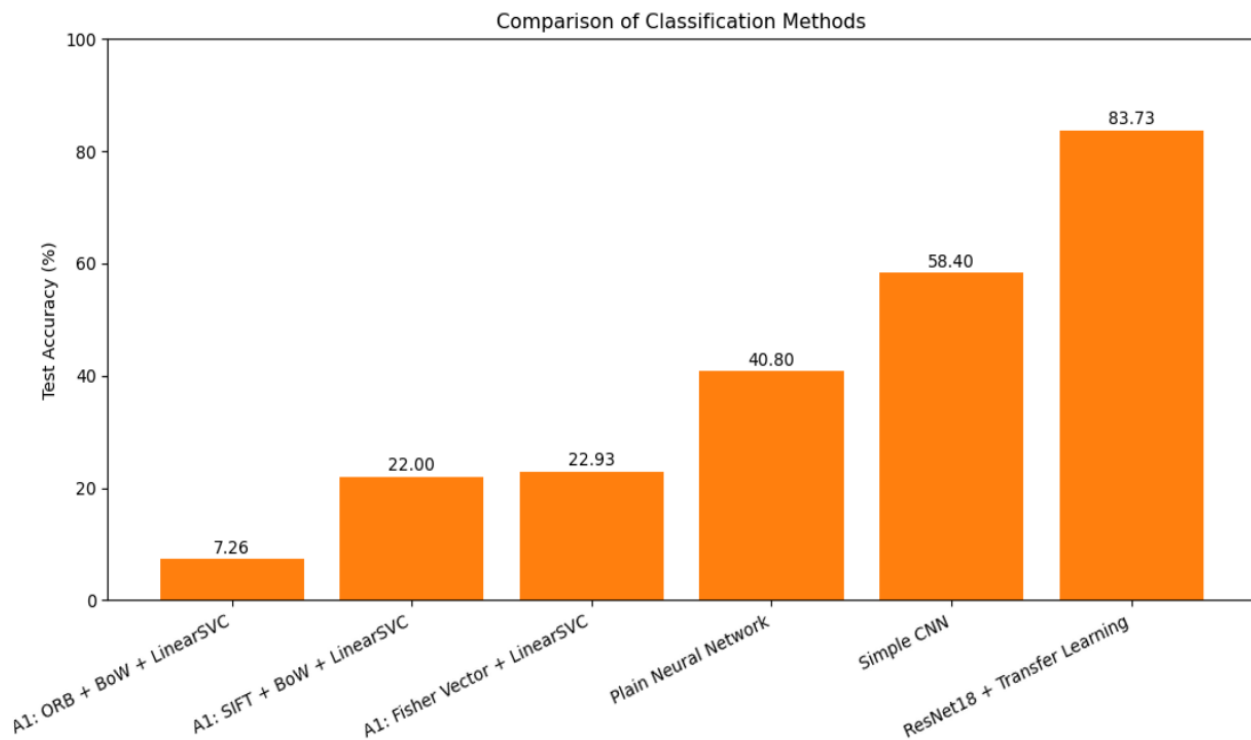
These results show deep learning methods, working better than traditional hand-crafted features. The plain neural network improved the accuracy by 17.87% compared to the best method for Project 1. Then, the simple CNN made the result 17.60% better than the plain neural network. ResNet18 with transfer learning made the biggest difference, increasing accuracy by 25.33% compared to the simple CNN.

Figure 1. Accuracy improvement between the main classification stages



A bar chart was also used to show the final test accuracies of all the methods in one picture. This made it easier to see how well each one worked. The chart showed that ResNet18 with transfer learning had the highest accuracy and the hand-crafted feature baseline had the lowest accuracy.

Figure 2. Accuracy comparison bar chart of all methods



To gain a more complete understanding of how each class performed, I created confusion matrices for the basic neural network, the simple CNN, and ResNet18. A confusion matrix displays the count of accurate and inaccurate predictions for each class, with values on the primary diagonal indicating correct predictions and off-diagonal values signifying errors. The confusion matrix for the basic neural network revealed a greater incidence of misclassifications across various classes, indicating the model's difficulty in distinguishing certain categories. Conversely, the diagonal of the straightforward CNN confusion matrix exhibited enhanced strength, signifying an increase in correct image classifications. The ResNet18 confusion matrix, in contrast, displayed the most pronounced diagonal and a reduced number of off-diagonal errors, thereby demonstrating its superior performance within the context of this project.

Figure 3. Confusion matrix of the plain Neural Network

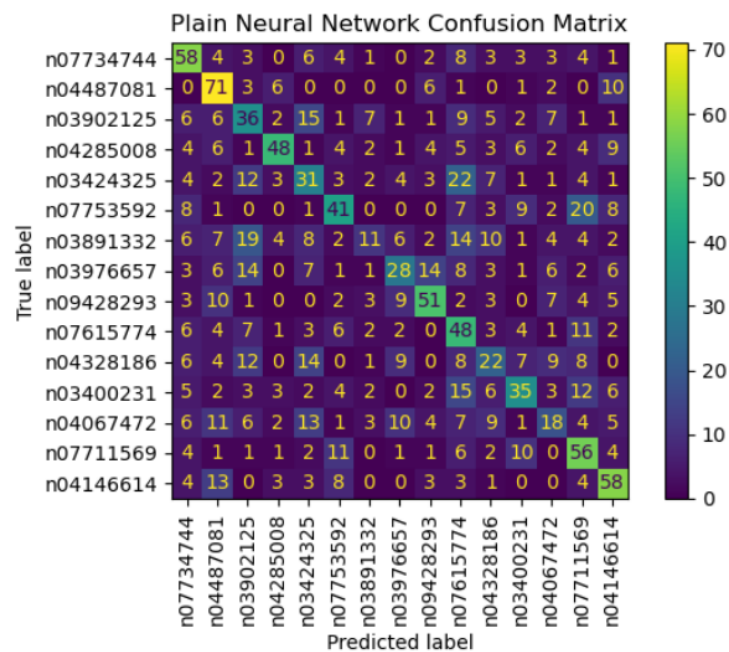


Figure 4. Confusion matrix of the simple CNN

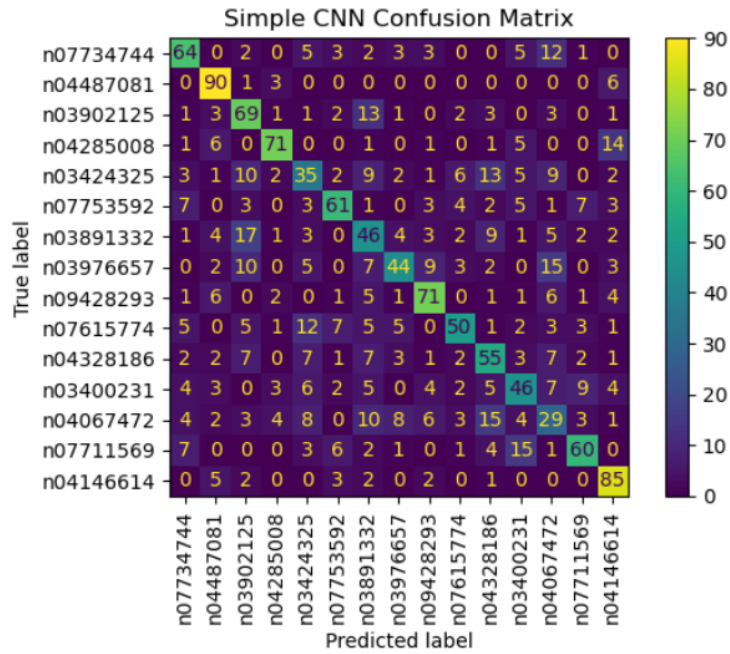
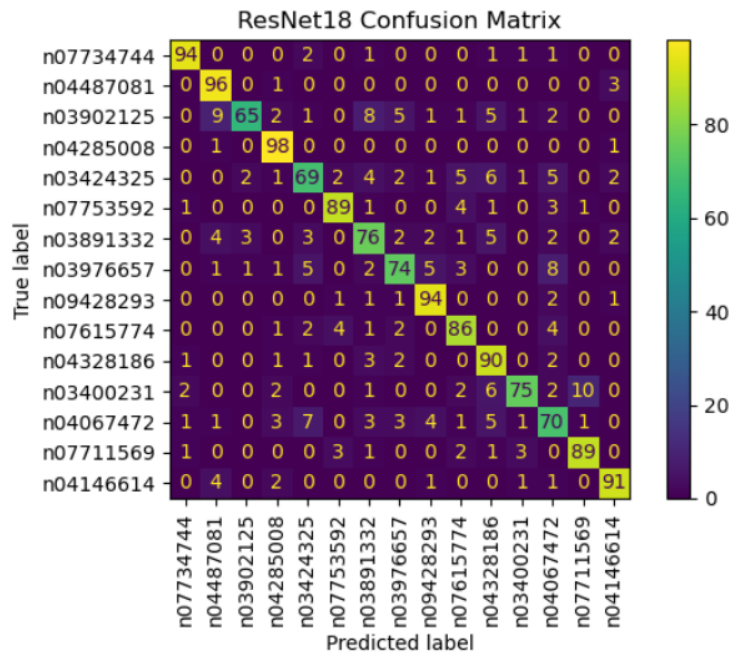


Figure 5. Confusion matrix of ResNet18 with transfer learning



In general, the results showed that accuracy went up at each main stage of the project. The hand-crafted method gave the worst result, the plain neural network made it better, the simple CNN made it even better, and ResNet18 with transfer learning gave the best final performance.

4. Discussion

The findings revealed a distinct disparity between the hand-crafted feature methodologies and the deep learning approaches. The most favorable outcome from Project 1, achieving an accuracy of 22.93%, was obtained using the Fisher Vector (SIFT + GMM) combined with LinearSVC. Following this, the basic neural network showed improvement, reaching an accuracy of 40.80%. The simple CNN then boosted performance, achieving an accuracy of 58.40%. Ultimately, ResNet18, using transfer learning, showed the best overall performance in this project, with an accuracy of 83.73%. Therefore, these results indicate that deep learning models performed better than manually created features in this image classification task.

A key difference arises from the use of hand-crafted features, which are based on manually created descriptors. In Project 1, I used methods like SIFT, ORB, Bag of Words, and Fisher Vector to describe the images before classifying them. Although these methods can still extract useful information from images, they are limited by their reliance on fixed feature designs, rather than learning directly from the data. The Fisher Vector method performed better than both ORB + BoW and SIFT + BoW in this project, suggesting that more detailed feature encoding improved the classifier's performance. But it still didn't do as well as the deep learning methods.

The plain neural network was better than the hand-made baseline, but it still had some big problems. The images were resized and then flattened into a single long vector before being put into the model. This method allowed the network to learn directly from pixel values, but it also removed the original image structure. As a result, the model couldn't easily use the spatial relationships between nearby pixels. Convolutional neural networks were created to better handle the 2D structure of images, which is why CNN-based methods often perform better in image-related tasks.

The basic CNN performed better than the regular neural network because it was better at classifying images. Initially, the Convolutional Neural Network (CNN) didn't flatten the image. Instead of changing the two-dimensional structure, the method used convolutional layers to find local patterns, such as shapes, edges, and textures. Pooling layers also helped reduce the image size gradually, while keeping important visual information. As a result, the basic CNN performed better than the regular neural network.

Transfer learning yielded the most favorable outcomes with ResNet18. ResNet architecture incorporates residual connections, which facilitate the training of deeper networks and streamline the optimization of complex architectures. Consequently, in numerous image recognition tasks, ResNet models demonstrate superior performance compared to conventional, custom-designed convolutional neural networks. Transfer learning also means that the model starts with what it learned from a big dataset and then changes that knowledge to fit the new task. This usually works better when the new dataset is smaller or when better performance is needed.

The outcomes were further influenced by the training methodology and parameter configurations. Various configurations were explored for the basic neural network, encompassing image dimensions, architectural design, and epoch count. A reduction in input size facilitated a decrease in the parameter count, thereby streamlining the model's training process. Furthermore, it was observed that excessive epoch utilization could diminish test performance, indicative of model overfitting. Regarding the simple CNN, data augmentation strategies, dropout regularization, and the choice of optimizer all contributed to enhanced generalization capabilities. Using pretrained weights and a lower learning rate proved beneficial for ResNet18, allowing it to learn effectively while preserving the value of its existing learned features.

The confusion matrices supported these findings. The basic neural network's confusion matrix showed more values off the diagonal, indicating that many classes were confused. In contrast, the simple CNN's confusion matrix had a stronger main diagonal, meaning it correctly classified more classes. The ResNet18 confusion matrix showed the clearest diagonal and made fewer mistakes in each class, which suggests it was the best-performing model in this study.

There were pros and cons to each method. The hand-crafted methods were simpler and easier to understand, but they didn't work as well. The plain neural network made the baseline better and was still simple, but flattening the image made it lose spatial information. The simple CNN was better for classifying images because it learned visual patterns directly from the structure of the images. However, it still needed careful tuning and didn't perform as well as ResNet18. The best result came from ResNet18 with transfer learning, but it was also the most advanced method and used a model that had already been trained instead of one that was built from scratch.

Also, the final hyperparameters were selected after testing different settings. In general, smaller input size helped the MLP, data augmentation improved generalization in the CNN-based models, and a smaller learning rate was more suitable for ResNet18 with transfer learning.

Overall, the results showed a clear improvement in performance. Going from hand-crafted features to a plain neural network made things better, going from the plain neural network to a simple CNN made things even better, and using ResNet18 with transfer learning gave the best result by a wide margin. This shows that the choice of model had a big impact on performance, and that deeper CNN-based methods worked best for this dataset.

Table 7. Advantages and disadvantages of the main methods

Method	Advantages	Disadvantages
<i>Hand-crafted methods</i>	Easy to understand and implement. Lower computational cost than deep learning models. Useful as a baseline.	Performance is limited because features are manually designed. Harder to capture complex image patterns. Lower accuracy than deep learning methods.
<i>MLP</i>	Better than hand-crafted methods because they learn from image data directly. Simpler than CNN models.	The image is flattened before training, so spatial information is lost. Less suitable for image classification than CNN.
<i>Simple CNN</i>	Learns visual patterns such as edges, textures, and shapes. Keeps image structure, so it is more suitable for image classification. Better accuracy than the MLP.	Needs more tuning and training time than the MLP. Performance is still lower than ResNet18 with transfer learning.
<i>ResNet18 + Transfer Learning</i>	Best accuracy in this project. Uses pretrained knowledge, so it learns more effectively. Stronger and deeper architecture than the simple CNN.	More complex than the other models. It depends on a pretrained model instead of being fully built from scratch.

5. Conclusion

In summary, this project demonstrated a clear improvement in image classification, moving from traditional, manually created features to deep learning methods. The best hand-crafted feature method from Project 1, which used Fisher Vector (SIFT + GMM) with LinearSVC, achieved an accuracy of 22.93%. A basic neural network model then improved this, reaching an accuracy of 40.80%. After that, the simple CNN performed better again and achieved 58.40% accuracy. The best result in the project was obtained by ResNet18 with transfer learning, which reached 83.73% accuracy.

These results show that model choice had a strong effect on performance. The hand-crafted methods were useful as a baseline, but they were weaker than the deep learning approaches. The plain neural network improved the baseline, but it still lost spatial information because the image was flattened before training. The simple CNN performed better because it could learn image features while keeping the image structure. Finally, ResNet18 with transfer learning gave the highest accuracy because it used stronger CNN architecture and pretrained knowledge.

The project's primary objective, which was to compare various image classification methods using the same dataset, was successfully accomplished. The findings indicated that CNN-based techniques demonstrated superior efficacy for this specific task, with transfer learning emerging as the most impactful strategy for enhancing performance. Furthermore, these results corroborated the central premise of the coursework, which involved constructing, assessing, and contrasting neural network and convolutional neural network methodologies on the same chosen dataset.

References

- Daniilidis, K., Maragos, P., & Paragios, N. (2010). Improving the Fisher Kernel for Large-Scale Image Classification. 156.
- Lowe, D. G. (2004). Distinctive Image Features. 28.
- Mitra, D. (2026, January 16). *Explainable Convolution Neural networks: Training ResNet-18 from scratch* . Retrieved from Medium:
<https://debayanmitra1993.medium.com/explainable-convolution-neural-networks-training-resnet-18-from-scratch-on-imagenet-200-dataset-3c745d3f1a98>
- Pykes, K. (2022, July 6). *PyTorch Tutorial: Building a Simple Neural Network From Scratch*. Retrieved from Datacamp: <https://www.datacamp.com/tutorial/pytorch-tutorial-building-a-simple-neural-network-from-scratch>
- Writer, C. (2025, Jun 23). *What Is ResNet-18?* Retrieved from Roboflow:
<https://blog.roboflow.com/resnet-18/>